

1 Musikgenerierung mit Transformern

1.1 Einleitung

1.2 Preprocessing

Für die Transformer-Architektur passen wir das Preprocessing ein wenig an. Jeden Choral erweitern wir um einen SOS-Token (Start-Of-Sequence) am Anfang und einen EOS-Token (End-Of-Sequence) am Ende. Alle diese modifizierten Choräle konkatenieren wir dann zu einem einzigen Vektor D .

Sei $C_i = [n_{i,1}, \dots, n_{i,\ell}]$ der i -ten Choral, der Datenvektor D ergibt sich dann als

$$D = [\text{SOS}, \underbrace{n_{1,1}, \dots, n_{1,\ell_1}}_{C_1}, \text{EOS}, \text{SOS}, \underbrace{n_{2,1}, \dots, n_{2,\ell_2}}_{C_2}, \text{EOS}, \dots, \text{EOS}]$$

Wir schreiben $D = [t_1, \dots, t_n]$ (wobei $t_1 = \text{SOS}$, $t_2 = n_{1,1}$ usw.), die einzelnen t s bezeichnen wir als Tokens.

Ein Input-Output-Paar (x, y) ist definiert als:

$$\begin{aligned} x &= [t_k, \dots, t_{k+\ell_{\text{SEQ}}-1}] \\ y &= [t_{k+1}, \dots, t_{k+\ell_{\text{SEQ}}}] \end{aligned}$$

Das Target y ist also die um eine Position verschobene Input-Sequenz.

Unter Berücksichtigung der drei Spezialtokens SOS, EOS und PAD erhalten wir eine Vokabulargröße V von 131.

1.3 Musikgenerierung mit Transformern

Der Generierungsprozess verläuft analog zu RNNs durch Sampling aus der Wahrscheinlichkeitsverteilung des Modells.

In unserem Fall gestalten wir die Generierung mit Transformern etwas flexibler: An die Anfangssequenz wird nur noch folgende Anforderung gestellt $[n_1, \dots, n_k]$, mit $0 \leq k \leq \ell_{\text{SEQ}}$ (wobei wir für $k = 0$ eine leere Liste meinen), das ist Konsequenz des SOS-Tokens; ℓ_{SEQ} ist dabei im Wesentlichen die maximale Sequenzlänge, die das Modell verarbeiten kann, wir wählen, falls nicht anders genannt, $\ell_{\text{SEQ}} = 512$.

Wie sich die Länge der Anfangssequenz auf die Vorhersage des Modells auswirkt, untersuchen wir in subsection 1.8.

Ebenso lassen wir das Modell "entscheiden", wann das generierte Stück endet; die Konsequenz des EOS-Tokens (um Endlosschleifen zu vermeiden, ist eine maximale Länge aber dennoch zu setzen).

Die Audiodateien finden sich unter folgenden Link: <https://www.kaggle.com/code/ferdibck/audiodateien-fuer-ausarbeitung-transformer>.

1.4 Die grundlegende Transformer-Architektur

Für die konkrete Struktur orientieren wir uns an Chollet, 2021, Da wir während der Inferenz nur Zugriff auf die bisherigen Noten haben, implementieren wir

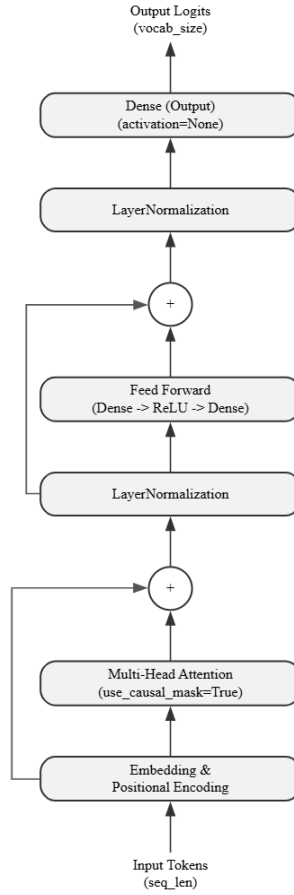


Figure 1: Grundlegende Struktur unseres Transformers

einen Decoder-Only-Transformer. Die in Figure 1 dargestellte Architektur lässt sich in folgende Komponenten unterteilen, wobei wir an entscheidenden Stellen Hyperparameter wählen müssen, die die Kapazität und das Lernverhalten des Modells beeinflussen:

- Wie im Ansatz mit RNNs werden die diskreten Input-Tokens in Vektoren der Dimension d_{emb} eingebettet. Um Informationen über die Positionen zu erhalten, addieren wir positionsabhängige Vektoren gleicher Dimension, in unserem Fall nutzen wir das in Vaswani et al., 2017 vorgestellte Positional-Encoding.
- Das Herzstück unseres Transformers ist natürlich die Multi-Head-Attention, die Anzahl der Attentions-Heads N_H können wir als weiteren Hyperparameter wählen. Entscheidend ist für uns vorallem die Maskierung: Da wir autoregressiv generieren, darf das Modell an Position t nicht keine

Informationen der noch zu generierenden Noten an $t + 1, \dots, \ell_{\text{SEQ}}$ erhalten.

- Nach der Attention-Schicht folgt ein einfaches Feed-Forward-Netzwerk, in unserem Fall bestehend aus zwei Schichten. Die Dimension innerhalb dieses Netzwerkes ist ein weiterer Hyperparameter d_{ffw} .
- Schließlich bildet die letzte Schicht auf einen Vektor der Dimension V ab, wir nutzen keine Aktivierungsfunktion, erhalten die sog. Logits als Output. Wendet man auf diese die Softmax-Funktion an, erhält man die Wahrscheinlichkeitsverteilung für das Noten-Sampling.

1.5 Ein erster Versuch

Layer (type)	Output Shape	Param #	Connected to
token_input (InputLayer)	(None, 512)	0	-
embedding (Embedding)	(None, 512, 64)	8,384	token_input[0][0]
add_6 (Add)	(None, 512, 64)	0	embedding[0][0]
attention (MultiHeadAttentio...	(None, 512, 64)	66,368	add_6[0][0], add_6[0][0]
add_7 (Add)	(None, 512, 64)	0	add_6[0][0], attention[0][0]
layer_normalizatio... (LayerNormalizatio...	(None, 512, 64)	128	add_7[0][0]
sequential_2 (Sequential)	(None, 512, 64)	16,576	layer_normalizat...
add_8 (Add)	(None, 512, 64)	0	layer_normalizat... sequential_2[0][...]
layer_normalizatio... (LayerNormalizatio...	(None, 512, 64)	128	add_8[0][0]
logits (Dense)	(None, 512, 131)	8,515	layer_normalizat...

Total params: 100,099 (391.01 KB)

Trainable params: 100,099 (391.01 KB)

Non-trainable params: 0 (0.00 B)

Figure 2: Modell-Zusammenfassung mit $d_{\text{embed}} = 64$, $N_H = 4$ und $d_{\text{ffw}} = 128$

Nun stehen wir nicht mehr ganz am Anfang, wir orientieren uns an der Idee, die im Falle von RNNs erfolgreich war; unser Modell soll also **eine** Note aus

$[0, 127]_{\mathbb{Z}}$ vorhersagen.

Gem. subsection 1.4 haben wir 3 Hyperparameter zu bestimmen, wir wählen diese analog zum RNN, siehe Figure 2:

- $d_{\text{embed}} = 64$
- $N_H = 4$
- $d_{\text{ffw}} = 128$

Wir trainieren ebenfalls für 20 Epochen mit EarlyStopping und ReduceLROnPlateau. Nach 15 Epochen wird das Training beendet und das beste Modell aus Epoche 10 übernommen: val-accuracy von ca. 84% und test-accuracy von ca. 82%. Wieder NICHT SCHLECHT. In Figure 3 findet sich das mithilfe des gleichen Input für RNNs generierten Musikstückes; Figure 4 zeigt hingegen ein vollständig generiertes Stück, nur mit dem SOS-Token als Input.



Figure 3: Generiertes Musikstück mit $\text{temp} = 1.2$, die ersten beiden Takte sind vorgegeben (A)

Trotz ein paar harmonischen Ungereimheiten ist das erste Stück bis Takt 20 für das ungeschulte Ohr recht gelungen, ab Takt 21 sind die generierten Akkorde eher der Sorte "Free Jazz". Doch auch die Transformer-Architektur leidet unter Wiederholung der Akkorde.

Das vollkommen selbst generierte Stück ist hingegen deutlich unharmonischer, so doch sich doch einige anhörliche Sequenzen finden lassen.

Doch auch hier sei gewarnt: Die zwei generierten Stücke haben wir exemplarisch ausgewählt, es geht natürlich auch schlechter.

1.6 Ein alter Bekannter

Wie oben gesehen hat der Transformer-Ansatz ebenfalls Probleme mit der Akkord-Harmonie, für RNNs hat hier ja die veränderte Loss-Funktion geholfen.



Figure 4: Generiertes Musikstück mit $\text{temp} = 1$, wir geben keine Noten vor (B)

Wir trainieren also auch unseren Transformer (Figure 2) mit diesen, wobei wir $w_1 = 0.01$, $w_2 = 0.1$ und $w_3 = 1$ wählen.

Nach 20 Epochen erhalten wir das beste Modell aus Epoche 19 mit val-acc von etwa 83% und test-acc von etwa 82%, von den reinen Werten ist also keine Verbesserung zu erkennen. In der Generierung lässt sich sogar eher eine Verschlechterung erkennen, im Eye-Test scheint diese instabiler zu sein, die generierten Stücke finden sich in Figure 5 und Figure 6.

Aufgrund der häufig vorkommenden Dissonanzen trainieren wir das Modell erneut, dieses mal mit $w_1 = 0.01$, $w_2 = 1$ und $w_3 = 10$. Das Ergebnis ist noch ernüchternder, das Modell beendet jedliche Notensequenzen mit dem EOS-Token nahezu sofort.

1.7 Ein Versuch von Data-Augmentation

Unser Trainings-Korpus ist mit 220 000 Tokens nahezu winzig im Vergleich zur heutigen Menge von Trainingsdaten für Transformer-Modelle. Vielleicht helfen auch in unserem Fall mehr Daten.

Das Augmenting funktioniert ganz einfach, für jeden Choral $C_i = [n_{i,1}, \dots, n_{i,\ell}]$, fügen wir auch $C_i + t = [n_{i,1} + t, \dots, n_{i,\ell} + t]$ in unseren Datenvektor D hinzu; wir verschieben die Noten einfach um t Halbtonschritte nach oben oder unten. Wir nutzen $t = -5, -4, -3, \dots, 6$.

Als weiteren Wermutstropfen erhalten wir auch mit Data Augmentation nur eine val-acc bei 84% und eine test-acc bei 83%. Die Generierung läuft allerdings wieder etwas stabiler als in subsection 1.6 (Zur Info: aufgrund der Trainingsdauer



Figure 5: Generiertes Musikstück mit $\text{temp} = 1.2$, die ersten beiden Takte sind wieder vorgegeben (C)



Figure 6: Generiertes Musikstück mit $\text{temp} = 1$, wir geben wieder keine Noten vor (D)

haben wir für dieses Modell $\ell_{\text{SEQ}} = 256$ gewählt).

1.8 Benchmarking

Abschließend wollen wir mit dem ersten Modell noch ein paar "Benchmarks" auf dem Test-Set durchführen.

Zunächst betrachten wir für eine fixe Input-Länge die Generierung als Klassifikationsproblem, Figure 7 zeigt die zugehörige Confusion-Matrix.

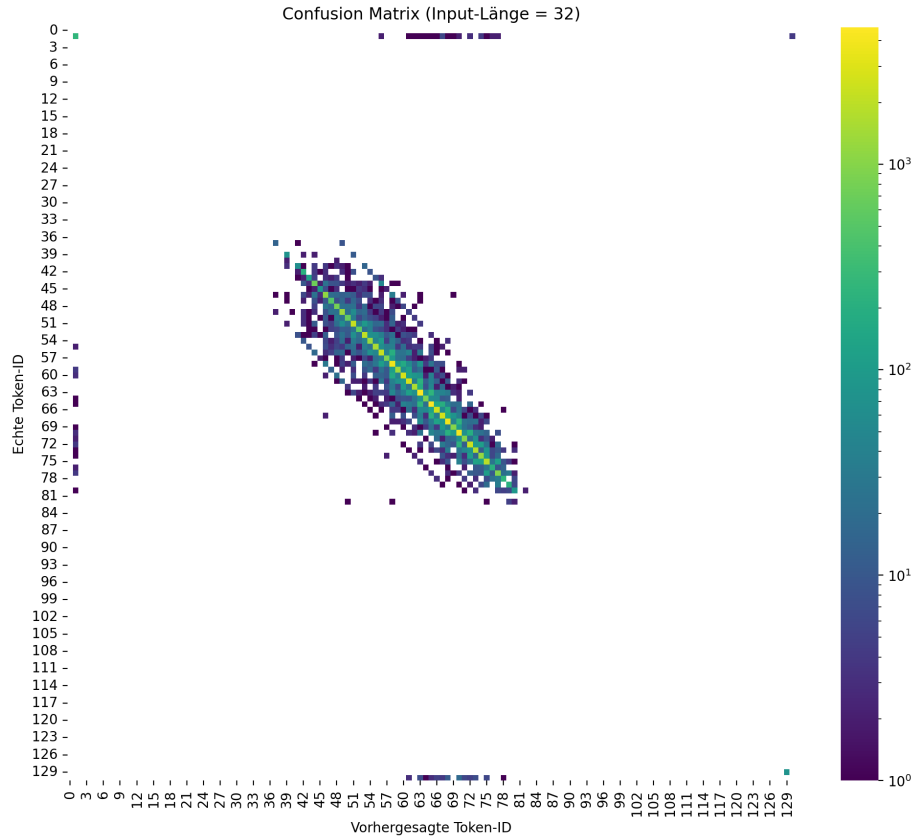


Figure 7: Confusion-Matrix, Input-Länge von 32

Uns fallen zwei Dinge auf zunächst werden wenige Tokens tatsächlich verwendet, das ist allerdings nicht verwunderlich; sehr tiefe bzw. hohe Tonhöhen kommen in Musik (jeglicher Zeit) kaum vor. Diesen "Beschränkung" hat auch unser Modell gelernt, ebenfalls dominiert die Hauptdiagonale die Vorhersagen deutlich, wir sagen also oft die korrekte Note vorher. Betrachtet man die Matrix genauer fallen die zwei symmetrischen Nebendiagonalen auf (die untere beginnt bei etwa (36, 48) und endet bei etwa (66, 78)), die Nebendiagonalen liegen also interessanterweise ± 12 Token-IDs unter bzw. über der Hauptdiagonale. Das sind genau 12 Halbtonschritte, also eine Oktave! Die Vorhersage ist also in diesem

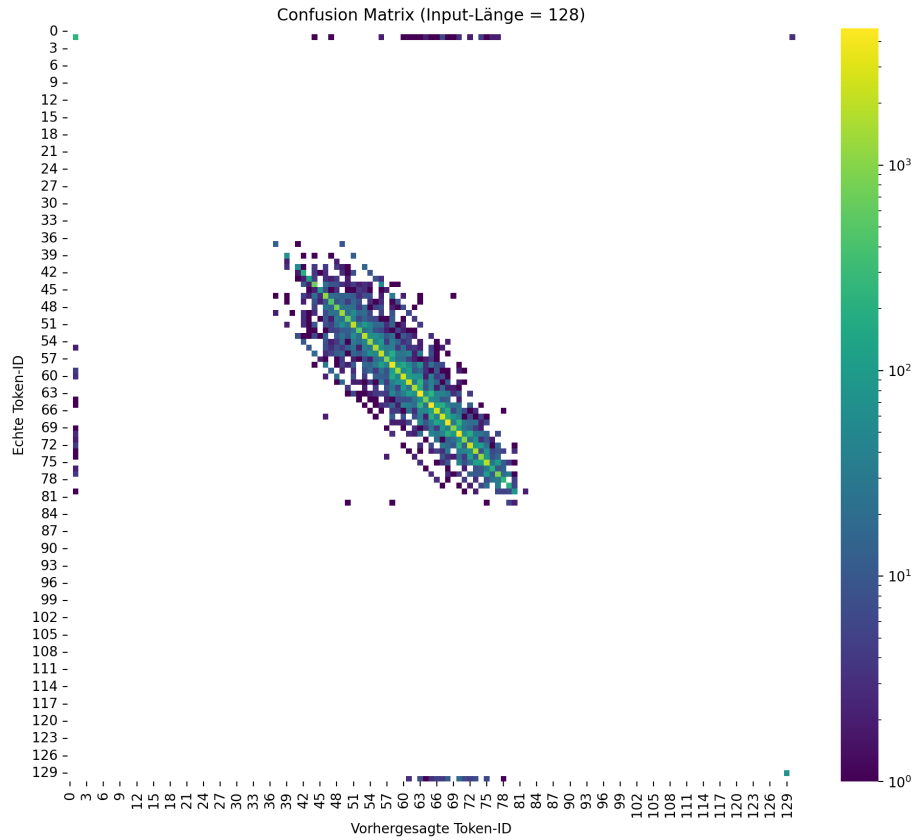
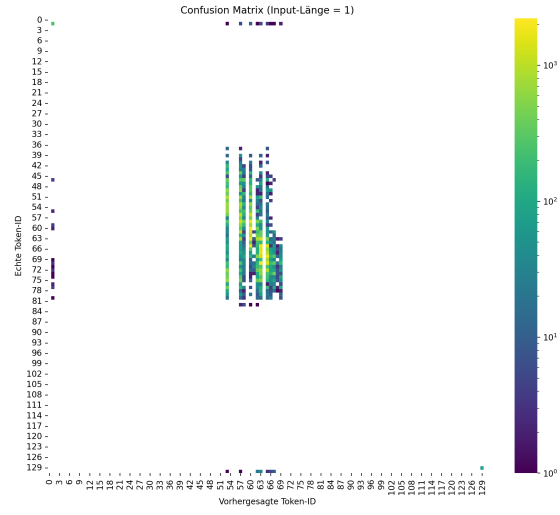


Figure 8: Confusion-Matrix, Input-Länge von 128

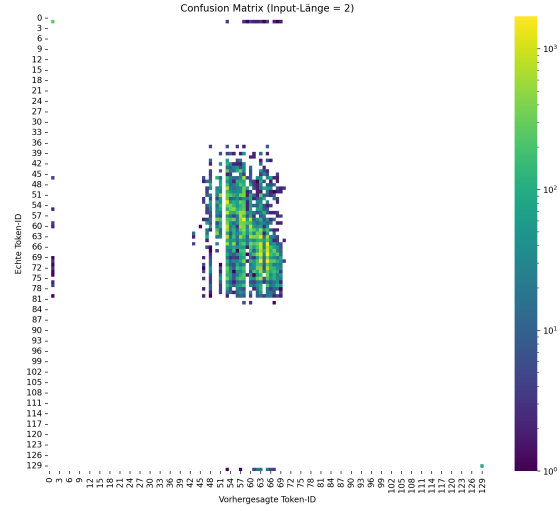
Fall harmonisch korrekt, wenn auch im falschen Intervall.

Figure 8 zeigt ein nahezu identisches Bild. Zum einen bestätigt das die Robustheit des Modells, doch ergibt sich daraus auch eine ernüchternde Konsequenz, Beziehungen von Noten über lange Zeiten sind nicht vorhanden oder werden vom Modell nicht gelernt; doch gerade darin haben Transformer-Modelle ihren Vorteil gegenüber anderen Methoden, siehe Vaswani et al., 2017. Das ist vermutlich auch der Grund für die nahezu identische Leistung gegenüber RNNs.

Figure 9a, Figure 9b, Figure 10a und Figure 10b zeigen die Entwicklung der Confusion-Matrizen mit länger werdender Input-Länge von 1, 2, 3, 4. Zusammen mit Figure 11 bestätigen diese die Beobachtungen; ab einer Input-Länge von 4 ist die Vorhersage des Modells praktisch fix, die Confusion-Matrix ist nahezu identisch zu den Matrizen mit Input-Länge von 32 und Input-Länge von 128. Das Modell profitiert also kaum von den Vorteilen der Transformerarchitektur.

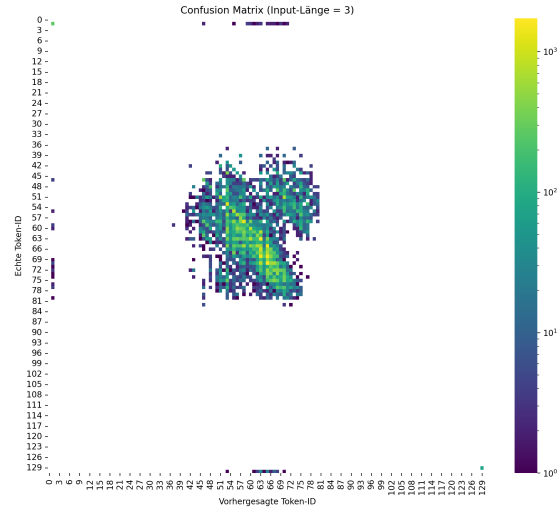


(a) Input-Länge von 1

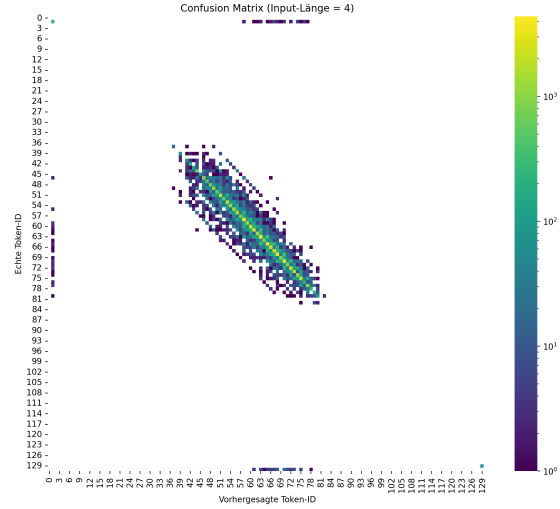


(b) Input-Länge von 2

Figure 9: Entwicklung der Confusion-Matrizen (Teil 1)



(a) Input-Länge von 3



(b) Input-Länge von 4

Figure 10: Entwicklung der Confusion-Matrizen (Teil 2)

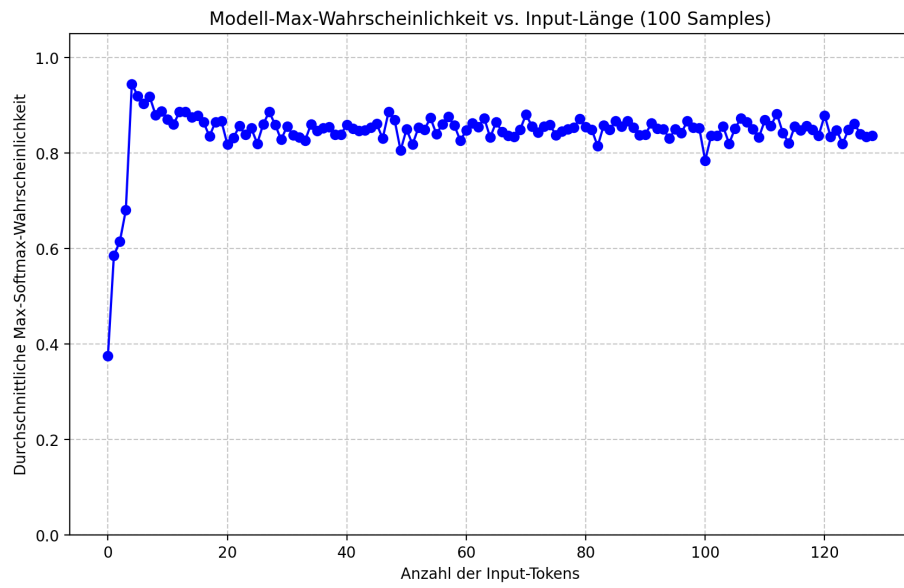


Figure 11: Maximale Wahrscheinlichkeiten für den nächsten Token in Abhängigkeit der Input-Länge